

ClaudeOS

AI Operating System — Client Handbook

Version 15.0 — Commercial Release

Prepared for: faiyke-ai

Date: 2026-05-27 | Edition: v15.0 Commercial

Confidential — For authorised users of ClaudeOS only.
Do not distribute outside your organisation.
Powered by faiyke-ai · ClaudeOS © 2026

Contents

#	Section	What You Will Find
1	Introduction	What ClaudeOS Is and What It Does
2	System Requirements	Hardware, Software, and Network
3	Installation & First-Run Setup	From Zero to Running in 15 Minutes
4	Logging In & First-Time Experience	Auth, Roles, Onboarding Modal
5	Dashboard Tour	Every Page Explained incl. Usage & Bg Colour
6	Working with AI Agents	Chat, Catalog, Run History
7	Memory System	How ClaudeOS Remembers Things
8	Workflows & Automation	Pipelines, Scheduling, Webhooks
9	Client Vault	Namespaces, Projects, Context Files
10	Outputs & Reports	Saving, Searching, Exporting
11	Ticketing System	Tickets + Email Notifications
12	Quality Scoring	LLM-as-Judge Automatic Evaluation
13	Observability	Quality, Latency, Token Cost, Memory Health, Namespace Usage (NEW)
14	Namespace Branding & Customisation	Per-Namespace Brand Config (NEW)
15	Custom Background Colors	Per-User Page Background (NEW)
16	Advanced Features	Images, Voice, MCP, A2A, Pulse Score

17	API Reference	REST API for Developers
18	Security Model	Auth, CORS, Rate Limits, Isolation
19	Cloud Sync	Supabase Backup
20	Troubleshooting & Support	Common Problems, Fixes, and Contacts

New in v14.0: Namespace White-Labeling (Sec 14), Custom Background Colors (Sec 15), Client Usage Dashboard with Pulse Score (Sec 5 & 16), Email Notifications for Tickets (Sec 11), Onboarding Modal (Sec 4), CORS & Rate Limiting (Sec 18), namespace-scoped KPIs, page URL persistence.

New in v15.0: Namespace Usage tab in Observability (Sec 13), Client Onboarding tab in Admin Panel (14-field schema seed), onboarding_done DB persistence (migration 018), admin context-aware unlock UI, voice widget reset on Clear Conversation, agent failure logging to global memory.

1 — Introduction

What Is ClaudeOS?

ClaudeOS is an AI Operating System — a secure, multi-user coordination layer that brings together AI agents, a long-term memory engine, automated workflows, project management, and a full ticketing system into one authenticated control centre. It runs as a private web application on your own infrastructure, so your data never leaves your environment.

Think of it as a team of 12 specialist AI assistants, all sharing the same memory, working from the same context, and all governed by the same security and quality rules — accessible from any browser. Version 14.0 adds per-namespace white-labeling, custom background colours, a dedicated client usage dashboard with a Pulse Score, and email notifications for tickets.

Core Capabilities

Capability	What It Does for You
AI Agents	12 specialist agents covering research, writing, analysis, scheduling, QA, client management, comms, and more. Run via chat, one-shot dispatch, or automated pipeline.
Long-Term Memory	Every fact, preference, reminder, and task you store is available to every agent. Memory persists between sessions and is scoped per namespace.
Workflow Automation	7 built-in pipelines (morning briefing, memory curation, research digest, client report, analysis run, QA sweep, meta-orchestration) on a schedule or on demand.
Ticketing	Full support and task lifecycle with email notifications — create, assign, prioritise, comment, resolve, and bulk-manage tickets without leaving ClaudeOS.
Quality Scoring	Every agent response is automatically scored on 4 dimensions by a fast AI judge.
Observability	Real-time dashboards for response quality, latency, token spend, and memory health.
Namespace Branding	Per-namespace company name, colours, icon, and background. Clients see their own brand.
Client Usage Dashboard	KPIs, Namespace Pulse Score, AI activity feed, and memory summary for client/viewer roles.

Architecture at a Glance

Browser

Streamlit Dashboard (:8501)

Login Gate (JWT) + Onboarding Modal

Flask REST API (:5000)

Auth & Roles (CORS, rate limits, security headers)

Agent Dispatcher (30/min run, 20/min stream)

Memory Engine (SQLite FTS5 + ChromaDB + hybrid BM25)

Workflow Scheduler (APScheduler + webhook triggers)

Output Manager

Ticketing + Email Notifications

Namespace Branding (per-namespace metadata)

Observability Optional: MCP Tool Server (:5100) – exposes agents to external AI clients

Note: ClaudeOS runs entirely on your own server. No agent prompts, memory entries, or outputs are sent to any third party except the Anthropic Claude API (for agent inference) and optionally Supabase (for cloud backup, if enabled).

2 — System Requirements

Server (where ClaudeOS runs)

Component	Minimum	Recommended
OS	Windows 10 64-bit	Windows 10/11 64-bit
CPU	2 cores	4+ cores
RAM	4 GB	8 GB+
Disk	10 GB free	20 GB+ SSD
Python	3.11	3.11 or 3.12
Internet	Required for Claude API calls	Stable broadband

Client Devices (browsers)

Any modern browser (Chrome, Edge, Firefox, Safari). No installation required on client devices. Mobile browsers are supported but a desktop screen is recommended for the Observability and Admin panels.

Required Accounts & Keys

Service	Purpose	Cost
Anthropic Claude API	Powers all AI agent inference. Required.	Pay-per-token (see anthropic.com/pricing)
SMTP Email Server	Email notifications for tickets. Optional.	Free (Gmail, Outlook) or paid SMTP relay
Supabase (optional)	Cloud backup of memory and outputs.	Free tier available

Important: Keep your `ANTHROPIC_API_KEY` secure. Never share it. If compromised, rotate it immediately in the Anthropic console and update your `.env` file.

3 — Installation & First-Run Setup

Step 1 — Get the Code

Copy the ClaudeOS project folder to your server. Top-level layout:

```
ClaudeOS/ ■■ agents/ ■■ core/ ■■ dashboard/ ■■ docs/ ■■ memory/ ■■ mcp/ ■■ scripts/ ■■ workflows/
■■ requirements.txt ■■ .env.example
```

Step 2 — Create Your Environment File

Copy `.env.example` to `.env` and fill in your values:

```
# .env ANTHROPIC_API_KEY=sk-ant-... # required FLASK_SECRET_KEY=change-this-secret # any long random
string FLASK_PORT=5000 STREAMLIT_PORT=8501 ALLOWED_ORIGINS=http://localhost:8501 # CORS whitelist
(comma-separated) # Optional – Email notifications SMTP_HOST=smtp.gmail.com SMTP_PORT=587
SMTP_USER=your@email.com SMTP_PASSWORD=your_app_password SMTP_FROM=no-reply@yourdomain.com # Optional –
Supabase cloud backup SUPABASE_URL=https://your-project.supabase.co
SUPABASE_SERVICE_KEY=your_service_role_key
```

Never commit `.env` to version control. The `.gitignore` already excludes it.

Step 3 — Install Python Dependencies

```
pip install -r requirements.txt
```

Installs Flask, Streamlit, Anthropic SDK, ChromaDB, sentence-transformers, APScheduler, waitress, plotly, PyJWT, bcrypt, and all other required packages.

```
# Optional – Voice input (~140 MB model on first use) pip install openai-whisper # Optional – MCP Tool
Server pip install mcp uvicorn
```

Step 4 — Initialise the Database

```
python scripts/migrate.py
```

Creates `data/claudeos.db` with all tables including auth, memory, agents, workflows, tickets, branding metadata, and observability columns.

Step 5 — Seed Default Data

```
python scripts/seed_agents.py # loads 12 agent definitions python scripts/seed_workflows.py # loads 7
default pipelines python scripts/seed_namespaces.py # creates the global namespace # Optional –
pre-populate client onboarding fields (14 fields, skips existing) python scripts/seed_client_schema.py
--namespace faiyke-ai
```

`seed_client_schema.py` creates blank placeholder memory entries for a client namespace (business name, industry, primary goals, brand tone, SLA tier, contact details, timezone, etc.). Agents read these automatically. Run after creating the namespace; safe to run multiple times.

Step 6 — Create the Admin Account

```
python scripts/create_admin.py --username admin --password Admin123!
```

Minimum password: 10 characters, at least one uppercase, one lowercase, one digit.

Step 7 — Start the System

```
.\scripts\start.ps1
```

Kills existing processes on ports 5000 and 8501, starts Flask via waitress, verifies `/health`, then starts Streamlit. After 5–10 seconds, open your browser to **[http://YOUR-SERVER-IP:8501]**.

Optional — Start the MCP Server

```
.\scripts\start_mcp.ps1
```

Stopping the System

```
netstat -ano | findstr :5000 # get PID taskkill /F /PID <PID> # kill it (repeat for 8501)
```

After any change to `.env`: always restart the server. Running processes do not pick up environment changes automatically.

4 — Logging In & First-Time Experience

First Login

Navigate to the dashboard URL in your browser. You will see the ClaudeOS login screen with two tabs: **Login** and **Register**. Your namespace branding (logo, colours, background) is applied to the login page automatically.

Enter the username and temporary password provided by your administrator, then press **Enter** or click **Login**. If the admin set the *must change password* flag, you will be prompted to set a new password immediately.

Password Requirements

- Minimum 10 characters
- At least one uppercase letter (A–Z)
- At least one lowercase letter (a–z)
- At least one digit (0–9)

Onboarding Modal (Client & Viewer Roles) NEW v14.0

The very first time a **client** or **viewer** logs in, a 5-slide onboarding modal appears automatically. It walks through the key features:

Slide	Title	What It Covers
1	Welcome	What ClaudeOS is, your namespace, and how to navigate.
2	AI Agents	How to run agents via chat and catalog. Streaming explained.
3	Memory	Writing memory entries, categories, and how memory helps agents.
4	Tickets	Creating and tracking tickets. Email notification opt-in.
5	Usage Dashboard	KPIs, Pulse Score, AI activity feed, and memory summary.

A progress bar and **Next / Dismiss** buttons control the modal. After dismissal, it does not appear again for that account. Admin and Operator roles do not see the onboarding modal.

Self-Registration (if enabled)

1. Click the **Register** tab on the login screen
2. Enter username, email, password, and select your namespace
3. Click **Register** — you are returned to the Login tab
4. Log in with your new credentials

User Roles

Role	What They Can Do
Admin	Full access — all namespaces, user management, branding config, admin panel, all features.
Operator	All namespaces and features except user management and the Admin panel.
Client	Own namespace only — Usage dashboard, memory, agents, outputs, tickets. Sees namespace branding on login.
Viewer	Own namespace, read-only. Cannot write memory or run agents.
Staff	Support role — only sees tickets assigned to them. Can self-assign open tickets.

Account Security

- **Account logout:** 5 failed attempts triggers a 15-minute logout.
- **Session tokens:** JWT access tokens expire after 60 minutes and are silently refreshed while you are active.
- **Case-insensitive usernames:** Alice and alice refer to the same account.
- **Page persistence:** your active page is saved in the URL query param `?page=PageName` — browser refresh returns you to the same page.

Theme Toggle

A dark/light mode button sits in the **bottom-left corner** of every page. Your theme preference persists for the current session.

5 — Dashboard Tour

Overview Page

The home page shows current system health and recent activity at a glance. KPIs are now **namespace-scoped**: admin/operator see global counts; client/viewer see only their namespace data.

Element	What It Shows
System Status strip	Green/red indicators for the API and database.
KPI Cards	Memory entries, registered agents, runs, workflows, outputs — scoped to your namespace.
Recent Events feed	Last 8 agent run results with agent name, namespace, timestamp, and quality score pill.
Quick Dispatch	Run any agent against any namespace with a one-off prompt.
Memory by Namespace	Count of active memory entries per namespace.
Auto-refresh toggle	Refreshes every 8 seconds — useful for monitoring live runs.
Error alert strip	Red banner when any recent run failed.
Running-now indicator	Yellow banner listing agents currently executing.

Eval Score Pills

Colour	Score Range	Meaning
	4.0 – 5.0	High quality.
	2.5 – 3.9	Acceptable — review before sending to client.
	0 – 2.4	Low quality — re-run with a more specific prompt.

Usage Page (Client & Viewer Only) NEW v14.0

A dedicated page for client and viewer roles providing a full activity overview for their namespace.

KPI Cards (30-day rolling)

KPI	Description
AI Runs (30d)	Total agent runs in the last 30 days for this namespace.
Tokens In / Out	Total input and output tokens consumed.

Est. Cost (USD)	Estimated spend based on current claude-sonnet-4-6 pricing.
Avg Quality	Rolling average eval score across all scored runs.
Outputs	Total saved outputs for this namespace.

Namespace Pulse Score

A composite 0–100 health score for the namespace, shown as a circular gauge.

Formula: (Avg Quality × 0.40) + (Ticket Resolution Rate × 0.30) + (Memory Freshness × 0.20) + (Workflow Success × 0.10)

Colour: ≥75 = Excellent / Good ≥50 = Fair <50 = Needs Attention No data = Getting Started

Recent AI Activity Feed

Last 10 agent runs for this namespace showing: agent name, status, eval score, duration, and timestamp.

Memory Summary

Total memory entries, fresh entries (written in the last 7 days), and date of last consolidation run.

Memory Page

- Full-text search across keys and values
- Filter by namespace, category, and minimum confidence
- Write entries with category, TTL, and confidence score
- Toggle **Hybrid Search** for BM25 + vector semantic retrieval
- Bulk delete via checkbox selection

Agents Page

Three tabs — Chat, Catalog, and Run History. Covered in detail in Section 6.

Workflows Page

7 pipelines, manual triggers, run history, and a Webhooks tab. Covered in Section 8.

Client Vault Page

Namespace stats, project list, context file upload. Covered in Section 9.

Outputs Page

All AI-generated content. Full-text search, date and type filters, per-output export. Covered in Section 10.

Tickets Page

Full ticket lifecycle with email notifications. Covered in Section 11.

Observability Page (Admin / Operator)

Five tabs: Quality Scores, Latency, Token Cost, Memory Health, and **Namespace Usage** (cross-namespace run/token/cost/quality comparison with stacked bar chart). Covered in Section 13.

Settings Page (Admin / Operator)

System config, Supabase sync controls, Email Notification settings, environment info.

Admin Panel (Admin only)

Six tabs: Users (context-aware unlock — Unlock button only shown for locked accounts), API Keys, Audit Log, Sessions, Security config, and **Client Onboarding** (fill 14 standard fields per namespace so agents have rich context from day one). Covered in Sections 14 and 18.

Page Persistence **NEW v14.0**

The active page is stored in the URL query parameter `?page=PageName`. Refreshing the browser returns you to the same page. Bookmarking a URL opens that page directly after login.

Background Colour Picker

The sidebar contains a **Background color** expander. Any logged-in user can set a custom page background. Covered in detail in Section 15.

6 — Working with AI Agents

The 12 Built-In Agents

Agent	Category	What It Does
Analysis Agent	Analysis	Structured reports, key findings, and recommendations from raw data or context.
Morning Briefing Agent	Ops	Synthesises overnight context into a daily briefing: reminders, client updates, priorities.
Client Manager Agent	Ops	Per-client project status cards with deadlines, blockers, and action items.
Communications Agent	Comms	Client-facing emails, messages, and proposals — tone-matched to each client.
Memory Curator Agent	System	Reviews, deduplicates, and consolidates memory entries.
Meta Orchestrator Agent	System	Decomposes complex goals into ordered chains of agent invocations.
QA Agent	Engineering	Code review against OWASP, platform compatibility, and brand compliance rules.
Research Agent	Research	Deep research with synthesis and source validation across a given topic.
Scheduling Agent	Ops	Converts natural language scheduling requests into memory reminder entries.
Transport Ops Agent	Domain	Domain-specific fleet and booking analysis — namespace-locked.
Workflow Builder Agent	System	Converts plain-English automation descriptions into ClaudeOS workflow YAML.
Writing Agent	Content	Drafts reports, emails, and proposals with voice matched to the active namespace.

Running an Agent — Three Ways

1. Chat (Recommended for iterative work)

Go to **Agents** → **Chat**. Select agent and namespace, type your request, press Enter. The response streams back in real time. Follow up with additional messages.

Best practice: be specific. “Write a 2-page executive summary of Q2 analysis results for the mobile app project, highlighting the top 3 risks” scores significantly higher than “write a report”.

2. Catalog (For one-shot tasks)

Go to **Agents** → **Catalog**. Find the agent, fill in prompt and namespace, click **Run**. Polled every 3 seconds until complete.

3. Quick Dispatch (From Overview)

The Overview page Quick Dispatch widget — select agent, namespace, enter prompt, run.

Namespace Scoping

Every agent run is scoped to a namespace. The namespace controls which memory entries are injected into context and where output is stored. Client and Viewer roles only see their own namespace.

Image Analysis

- Attach a screenshot, chart, or document for visual analysis in the Chat tab
- Agent receives the image as a content block alongside your prompt
- Uses: dashboard analysis, UI review, chart interpretation, document extraction

Voice Input

Click the microphone in the Chat tab. Record your request (up to 30 seconds). ClaudeOS transcribes locally using Whisper — nothing sent to external services. Review transcription, edit if needed, then submit. Requires `pip install openai-whisper`.

Understanding Token Usage

Each response shows input tokens (context sent) and output tokens (response generated). To keep costs low:

- Use specific prompts (less context padding needed)
- Clear conversation history when starting a new topic
- Use lighter agents for simple tasks (e.g. Scheduling Agent vs. Research Agent)

Rate Limits

Endpoint	Limit
Agent <code>/run</code>	30 requests per minute per IP
Agent <code>/stream</code>	20 requests per minute per IP
Workflow <code>/trigger</code>	10 requests per minute per IP

If a rate limit is hit, the API returns `429 Too Many Requests`. Wait and retry.

7 — Memory System

What Is Memory?

Memory is ClaudeOS's persistent knowledge store. When you or an agent writes a memory entry, it is stored in the database and automatically injected into every subsequent agent run in that namespace. Agents remember facts, preferences, context, and reminders across sessions — without you having to repeat yourself.

Memory Categories

Category	Use For	Example
Fact	Stable information about a client or project	"Client prefers invoices in GBP"
Preference	Tone, style, or format preferences	"Reports should start with an executive summary"
Reminder	Time-sensitive items that expire automatically	"Follow up on proposal by 2026-06-01"
Task	Action items for agents or team members	"Prepare Q3 analysis before board meeting"

Writing a Memory Entry

- Go to **Memory** page → click **Add Entry**
- Fill in: key (short identifier), value (content), category, TTL (optional expiry in days), confidence (0–1)
- For reminders: use the date/time picker to set the exact expiry
- Click **Save**

Searching Memory

Full-text search across keys and values. Toggle **Hybrid Search** for BM25 + semantic vector retrieval — better recall for conceptual queries (e.g. "client payment terms" finds entries about invoicing even without exact keyword match).

How Memory Is Injected into Agents

When an agent runs, ClaudeOS retrieves the most relevant memory entries using hybrid search and injects them into the agent's system prompt as structured context. The tiered context system reduces token usage by ~40% versus injecting all memory.

Memory Consolidation

An automated job runs every 4 hours. It finds groups of similar entries, uses Claude to synthesise each cluster into one concise entry, and archives the originals. Original entries are never hard-deleted — they are archived (`archived=1`).

Trigger consolidation immediately: **Observability** → **Memory Health** → **Trigger Consolidation**.

Tip: write memory entries in full sentences. “The client's primary contact is James Brown, james@example.com, +44 7700 900000” is far more useful to an agent than “james contact”.

8 — Workflows & Automation

What Are Workflows?

Workflows are multi-step AI pipelines that chain agent calls, passing output from one step as input to the next. They run on a cron schedule or are triggered manually or via webhook.

Built-In Workflows

Workflow	What It Does	Default Schedule
Morning Briefing	Synthesises overnight memory into a daily briefing.	Mon–Fri 07:00
Memory Curation	Deduplicates and consolidates memory entries.	Every 4 hours
Research Digest	Research Agent deep-dive and structured digest.	On demand
Client Report	Client Manager Agent generates status cards for all active projects.	Weekly
Analysis Run	Analysis Agent processes queued data and produces findings.	On demand
QA Sweep	QA Agent reviews recent outputs against quality and compliance rules.	On demand
Meta-Orchestrate	Meta Orchestrator decomposes a complex goal into a chain of agent calls.	On demand

Running a Workflow Manually

1. Go to **Workflows** page
2. Find the workflow → click **Run Now**
3. The run appears in Recent Events on the Overview page

Webhook Triggers

Any workflow can be triggered by an external system without a login token. Rate limit: **10 requests per minute** per IP.

Enable a Webhook

1. Workflows page → select workflow → **Webhooks** tab
2. Click **Enable Webhook** — a unique URL and HMAC secret are generated
3. Copy both. The secret is shown once — store it safely.

Fire the Webhook

```
curl -X POST [http://YOUR-SERVER-IP:5000]/api/v1/workflows/morning-briefing/trigger \ -H  
"X-Webhook-Secret: your-secret" \ -H "Content-Type: application/json" \ -d '{"context": {"namespace":  
"global", "topic": "AI news"}}'
```

Viewing Workflow Run History

Each workflow card shows its last run timestamp and status. Expand to see the step-by-step run log.

9 — Client Vault

Namespaces

A namespace is a fully isolated workspace. All memory, outputs, and agent context is scoped per namespace — nothing leaks between namespaces. The Client Vault page shows:

- Namespace name, description, and icon (set in Admin → Branding)
- Workspace statistics: number of files, total size in KB
- Number of active projects

Projects

- List all projects in your namespace
- Update a project's status: **Active** / **Paused** / **Archived**
- See project descriptions and creation dates

Context Files

Documents uploaded per namespace that are automatically injected into every agent system prompt for that namespace. Use for:

- Brand guidelines and tone-of-voice documents
- Standard operating procedures
- Client briefs or background documents
- Technical specifications that agents should always be aware of

Supported formats: plain text, Markdown, PDF.

Note: Keep context files concise. Very large files increase token usage on every agent call. Split large documents into smaller topical files.

10 — Outputs & Reports

What Are Outputs?

Every time an agent produces a response with `save_output: true` (the default), the full output is saved to the database and filesystem, auto-tagged with agent name, namespace, output type, and timestamp.

Browsing Outputs

- Search by keyword across all output content (full-text)
- Filter by namespace, output type (report / summary / plan / analysis / code / other), and date range
- Click any output to expand and read the full content

Exporting an Output

Format	Notes
Markdown	Raw text with Markdown formatting. Opens in any text editor. Always available.
PDF	Rendered PDF with ClaudeOS branding. Requires wkhtmltopdf installed on the server.

Bulk Delete

Select multiple outputs with checkboxes and click the bulk delete button. Permanent — not recoverable unless Supabase cloud sync is enabled.

Tip: use type and date filters to find old or low-quality outputs before bulk deleting.

11 — Ticketing System

Overview

The ticketing system provides a full support and task lifecycle inside ClaudeOS. Version 14.0 adds **email notifications** for ticket events.

Priority / SLA Tiers

Priority	Label	Meaning
P1	Critical	System down or blocking. Immediate response required.
P2	High	Major issue, significant business impact.
P3	Medium	Important but not blocking.
P4	Low	Nice-to-have, enhancement, or minor issue.

Ticket Lifecycle

Open → In Progress → Resolved → Closed

- **Open:** created, not yet assigned or started
- **In Progress:** assigned and being worked on
- **Resolved:** work complete, awaiting confirmation
- **Closed:** confirmed resolved, resolution note recorded

Creating a Ticket

1. Go to **Tickets** page → click **New Ticket**
2. Fill in title, description, priority, namespace, and optionally assign users
3. Click **Create**

Assignment

- **Admin / Operator:** can assign any ticket to any user
- **Staff:** can self-assign any open ticket
- **Client / Viewer:** cannot assign tickets

Email Notifications NEW v14.0

ClaudeOS sends automatic email notifications for key ticket events when SMTP is configured. Configure in **Settings** → **Email Notifications**.

Configuring SMTP

Setting	Description
SMTP Host	Your email server (e.g. <code>smtp.gmail.com</code>)
SMTP Port	Usually 587 (TLS) or 465 (SSL)
SMTP User	Email address used for sending
SMTP Password	App password or SMTP credential
From Address	Displayed sender (e.g. <code>no-reply@yourcompany.com</code>)

Notification Events

Event	Who Receives It
Ticket created with assignees	All assigned users receive a notification email
Ticket resolved or closed	Ticket creator receives a resolution email
SLA breach	Assignees and admin receive an SLA alert email

Global Toggle & Test Send

A **global enable/disable toggle** controls all email notifications at once. Use the **Test Send** button to verify your SMTP configuration is working before enabling live notifications.

Gmail users: use an App Password (not your account password) if 2-factor authentication is enabled. Generate one at `myaccount.google.com` → Security → App Passwords.

Comments

Expand a ticket and click the **Comments** toggle to load threaded comments. Comments load on demand to keep the page fast.

Resolution Notes & Bulk Delete

Record what was done in the resolution note field when moving a ticket to Resolved or Closed. Bulk delete is available to Admin / Operator only.

Stats Panel

Ticket counts broken down by status and priority at the top of the Tickets page.

12 — Quality Scoring

How It Works

Every agent run is automatically scored by Claude Haiku (fast, low-cost) on 4 dimensions. Scoring happens asynchronously in the background — no added latency. Scores appear 5–15 seconds after a run completes.

Scoring Dimensions

Dimension	Scale	Weight	What It Measures
Task Completion	0–5	40%	Did the output fully address the prompt?
Factual Grounding	0–5	30%	Are claims grounded in the injected memory context? Penalises hallucination.
Conciseness	0–5	20%	Is the output the right length? Penalises padding and repetition.
Safety	pass/fail	10%	Does output contain harmful content? Fail caps overall score at 1.0.

Overall score = (Task × 0.40) + (Factual × 0.30) + (Concise × 0.20) + (Safety × 0.10)

Where Scores Appear

- **Run History:** colour-coded pill per run
- **Overview:** pill on each recent event
- **Chat:** badge below each assistant response (~10s delay)
- **Usage Dashboard:** 30-day average in Avg Quality KPI card
- **Observability** → **Quality Scores:** aggregated analytics

Improving Scores

Low score on...	What to do
Task Completion	Be more specific. State exactly what you want, the format, and the desired length.
Factual Grounding	Add more relevant facts to Memory before running the agent.
Conciseness	Add “be concise” or “maximum 3 paragraphs” to your prompt.
Safety	Review the output for any problematic content. Adjust the prompt.

13 — Observability

Available to **Admin** and **Operator** roles only. Five sub-tabs.

Quality Scores Tab

- Per-agent average eval scores in a ranked table
- Score distribution chart (0.5-step buckets across all runs)
- Dimension breakdown: task completion, factual grounding, conciseness, safety
- Red alert when any agent's average drops below 2.5

Latency Tab

- p50, p95, and p99 latency across all runs
- Per-agent average latency table sorted by slowest
- Time-series chart over the past 7 days

Response length	Typical latency (claude-sonnet-4-6)
Short (under 200 tokens)	2–6 seconds
Medium (200–800 tokens)	6–20 seconds
Long (800+ tokens)	20–60 seconds

Token Cost Tab

- Total input and output token counts across all runs
- Estimated USD cost based on current claude-sonnet-4-6 pricing
- Per-agent breakdown: tokens used, cost, run count, average cost per run

Token cost estimates are approximate based on list pricing at ClaudeOS release. Check Anthropic's pricing page for current rates.

Memory Health Tab

- Entry counts per namespace: total, active, consolidated, expired
- Storage size estimate per namespace
- **Trigger Consolidation** button — runs the consolidation job immediately
- Visual indicator when any namespace has not been consolidated in over 24 hours

Namespace Usage Tab NEW v15.0

Cross-namespace comparison for admins and operators. Shows, for each namespace:

- Total agent runs, input tokens, output tokens, estimated USD cost
- Average quality score and memory entry count

- Stacked bar chart (plotly) comparing token consumption across all namespaces

Based on up to 500 most recent runs. Helps identify which namespaces are most active and where token spend is concentrated.

14 — Namespace Branding & Customisation NEW v14.0

Overview

ClaudeOS supports full per-namespace white-labeling. Each namespace can have its own company name, icon, primary colour, accent colour, and background colour. Clients see their own brand applied to the login screen, sidebar, and dashboard headers — creating a personalised experience without any code changes.

Configuring Namespace Branding

Only the **Admin** role can configure branding. Go to **Admin Panel** → **Branding** tab.

Available Brand Settings

Setting	Description	Example
Company Name	Replaces “ClaudeOS” in the sidebar header for users of this namespace	Acme Corp AI
Icon Emoji	Emoji shown next to the company name in the sidebar	■ ■ ■
Primary Color	Used for buttons, active highlights, headings, and progress bars	#1565C0 (blue)
Accent Color	Used for hover states, secondary actions, and links	#42A5F5 (light blue)
Background Color	Default page background for all users in this namespace. Overrideable per-user (Sec 15).	#F0F4FF

Live Preview

The Branding tab shows a live preview panel as you adjust settings. Sample text, a button, and a card are rendered using the chosen colours with auto-computed contrast text, so you can verify readability before saving.

Auto-contrast: ClaudeOS automatically derives readable text colours from your chosen primary and background colours using the WCAG luminance formula. You cannot set a colour combination that fails basic contrast requirements — the UI will warn you if a colour is too similar to the background.

Where Branding Appears

- **Login screen:** company name, icon, and background colour applied before login
- **Sidebar:** company name and icon replace “ClaudeOS” header
- **Dashboard headers:** primary colour used for heading accents
- **Buttons and highlights:** accent colour used for interactive elements

White-Label Footer

All white-labeled deployments show a subtle “*Powered by ClaudeOS*” note in the page footer. This attribution cannot be removed in the standard licence.

Background Color for Namespace

The background colour set in the Branding tab becomes the default for all users in that namespace. Individual users can override it using the personal background colour picker (see Section 15). The namespace default persists across sessions; the personal override is session-only and resets on logout.

Resetting to Default

Click **Reset to Defaults** in the Branding tab to restore ClaudeOS green (#407E3C) and white for that namespace. This takes effect on the next page load for all users in the namespace.

15 — Custom Background Colors NEW v14.0

Overview

Any logged-in user can change the page background colour for their current session using a colour picker in the sidebar. Text, surface, and border colours are automatically derived from the chosen background to ensure readability.

How to Set a Custom Background

1. Log in and look at the left sidebar
2. Find the **Background color** expander and click to expand it
3. Check the **Use custom background** checkbox — a colour picker appears
4. Choose any colour using the picker
5. The page background updates immediately with auto-derived surface and text colours

Auto-Derived Colour Logic

ClaudeOS uses the WCAG relative luminance formula to compute readability:

Background luminance	Derived text colour	Derived surface colour
Light background (luminance > 0.5)	#1A1A1A (near-black)	Slightly darker shade of background
Dark background (luminance ≤ 0.5)	#E8F5E8 (light green-white)	Slightly lighter shade of background

This ensures that regardless of the chosen colour, text remains readable and UI surfaces remain visually distinct from the page background.

Priority Order

#	Source	Notes
1	Personal override	Colour chosen by the user in the sidebar picker. Session-only.
2	Namespace default	Background colour set by admin in Admin → Branding. Persists for all namespace users.
3	Theme default	Standard dark/light theme background (“Dim” dark or white light).

Resetting to Default

Uncheck **Use custom background** in the sidebar expander. The background reverts to the namespace default (if set) or the current theme default.

Session Scope

Personal background colour choices are **session-only**. They reset when you log out or close the browser. The namespace background (set by admin) persists permanently until the admin changes it.

Accessibility tip: choose backgrounds with sufficient contrast from your primary colour. Avoid mid-range greys that create ambiguity for both light-text and dark-text rendering. Saturated pastels and deep rich tones tend to work best.

16 — Advanced Features

Multi-Turn Conversation History

Each agent in Chat mode maintains its own conversation session stored in the database. Refreshing the page does not lose the conversation within the same session. Click **Clear Conversation** when moving to a new topic to avoid stale context.

Image and Screenshot Analysis

In the Chat tab, attach any image file. The image is encoded as a base64 content block and sent alongside your prompt to Claude's vision API. Practical uses:

- Summarise or critique a report screenshot
- Interpret chart trends with the Analysis Agent
- Quick UI compliance check with the QA Agent
- Extract data from document photos

Voice Input

Requires `pip install openai-whisper`. First use downloads ~140 MB model. Subsequent starts are fast (model cached in memory). All transcription is done locally.

1. Click the microphone button in the Chat tab
2. Record your request (up to 30 seconds recommended)
3. Review the transcription in the prompt field, edit if needed, then submit

Namespace Pulse Score

The Pulse Score (shown on the Usage page for client/viewer roles) gives a 0–100 composite health indicator for a namespace.

Formula:

$$\begin{aligned} &(\text{Average eval quality} \div 5) \times 100 \times 0.40 \\ &+ (\text{Ticket resolution rate}) \times 100 \times 0.30 \\ &+ (\text{Memory freshness score}) \times 100 \times 0.20 \\ &+ (\text{Workflow success rate}) \times 100 \times 0.10 \end{aligned}$$

Colour thresholds: ≥75 Excellent/Good ≥50 Fair <50 Needs Attention No data: Getting Started

MCP Tool Server

The Model Context Protocol (MCP) server exposes all 12 ClaudeOS agents as native tools to any MCP-compatible AI client — Claude Desktop, Cursor, or custom agents.

```
.\scripts\start_mcp.ps1 # starts on port 5100 curl http://localhost:5100/mcp # verify it is running
```

Connecting Claude Desktop

Add to `%APPDATA%\Claude\claude_desktop_config.json`:

```
{ "mcpServers": { "claudeos": { "url": "http://localhost:5100/mcp" } } }
```

Restart Claude Desktop. All 12 agents appear as available tools in the tool picker.

A2A Agent Cards

Each agent exposes a machine-readable capability card for agent-to-agent discovery:

```
GET [http://YOUR-SERVER-IP:5000]/api/v1/agents/writing-agent/.well-known/agent.json
```

The card describes name, description, input/output schema, and streaming / multi-turn capabilities. External AI orchestrators use this to auto-discover and delegate to ClaudeOS agents.

17 — API Reference

The ClaudeOS REST API runs at `[http://YOUR-SERVER-IP:5000]/api/v1/`. All endpoints require authentication (JWT Bearer or X-API-Key) except `/health` and webhook triggers. Rate limits are enforced on all agent and workflow endpoints (see Section 6).

Authentication

JWT (Interactive Users)

```
# 1. Login curl -X POST [http://YOUR-SERVER-IP:5000]/api/v1/auth/login \ -H "Content-Type: application/json" \ -d '{"username":"your_user","password":"your_pass"}' # Response { "access_token": "eyJ...", "refresh_token": "eyJ...", "user": { "id": 1, "username": "your_user", "role": "operator" } } #
2. Use the access token curl [http://YOUR-SERVER-IP:5000]/api/v1/agents \ -H "Authorization: Bearer eyJ..."
```

Access tokens expire after 60 minutes. Refresh:

```
curl -X POST [http://YOUR-SERVER-IP:5000]/api/v1/auth/refresh \ -H "Authorization: Bearer <refresh_token>"
```

API Key (Scripts and Automation)

```
curl [http://YOUR-SERVER-IP:5000]/api/v1/memory \ -H "X-API-Key: your_api_key"
```

Core Endpoint Reference

Method	Endpoint	Description
POST	<code>/auth/login</code>	Login, returns access + refresh tokens
POST	<code>/auth/refresh</code>	Renew access token
POST	<code>/auth/logout</code>	Revoke current session
GET	<code>/auth/me</code>	Current user info and role
GET	<code>/health</code>	Public health check (no auth)
GET	<code>/memory</code>	List memory entries (filterable)
POST	<code>/memory</code>	Write a memory entry
DELETE	<code>/memory/<id></code>	Delete a memory entry
GET	<code>/memory/hybrid-search</code>	Hybrid BM25+vector search with RRF reranking
POST	<code>/memory/consolidate</code>	Trigger memory consolidation immediately
GET	<code>/agents</code>	List all registered agents

POST	<code>/agents/<name>/run</code>	Dispatch async agent run — 30/min limit
GET	<code>/agents/runs/<id></code>	Poll run status and output
GET	<code>/agents/runs</code>	List all runs (filterable)
GET	<code>/agents/<name>/stream</code>	SSE streaming — 20/min limit
GET	<code>/agents/<name>/.well-known/agent.json</code>	A2A Agent Card
GET	<code>/outputs</code>	List outputs
DELETE	<code>/outputs/bulk</code>	Bulk delete outputs
GET	<code>/tickets</code>	List tickets
POST	<code>/tickets</code>	Create a ticket (triggers email if configured)
PUT	<code>/tickets/<id></code>	Update ticket (triggers email on resolve/close)
DELETE	<code>/tickets/bulk</code>	Bulk delete tickets
GET	<code>/tickets/stats</code>	Ticket counts by status and priority
GET	<code>/workflows</code>	List workflows
POST	<code>/workflows/<name>/run</code>	Trigger workflow (JWT required)
POST	<code>/workflows/<name>/trigger</code>	Webhook trigger (X-Webhook-Secret, 10/min limit)
POST	<code>/workflows/<name>/webhook/enable</code>	Enable webhook, generate secret
GET	<code>/system/status</code>	System health detail
GET	<code>/system/namespace-stats</code>	Per-namespace KPIs for Usage dashboard
GET	<code>/admin/branding/<namespace></code>	Get namespace branding config (admin)
PUT	<code>/admin/branding/<namespace></code>	Update namespace branding (admin)

Async Run — Full Example

```
# Dispatch
curl -X POST [http://YOUR-SERVER-IP:5000]/api/v1/agents/writing-agent/run \
-H "Authorization: Bearer <token>" \
-H "Content-Type: application/json" \
-d '{ "prompt": "Write a 1-page executive summary of Q2 results", "namespace": "my-namespace", "save_output": true }' # Response 202
{"run_id": "abc123", "status": "pending", "poll_url": "/api/v1/agents/runs/abc123"} # Poll until done
curl [http://YOUR-SERVER-IP:5000]/api/v1/agents/runs/abc123 -H "Authorization: Bearer <token>" #
Response when complete { "run_id": "abc123", "status": "done", "output": "Q2 Executive Summary\n...",
"tokens_in": 1420, "tokens_out": 380, "eval_score": 4.2 }
```

SSE Streaming — Full Example

```
curl -N "[http://YOUR-SERVER-IP:5000]/api/v1/agents/writing-agent/stream?prompt=Hello&namespace=my-ns" \
-H "X-API-Key: your_api_key" data: {"type": "token", "text": "Hello"} data: {"type": "token", "text": "
there"} data: {"type": "done", "run_id": "xyz", "tokens_in": 900, "tokens_out": 12} data: {"type":
"error", "message": "Rate limit exceeded"}
```

18 — Security Model

Authentication

Mechanism	Details
JWT access tokens	60-minute TTL. Silently refreshed while active. Never stored in plaintext.
Refresh tokens	7-day TTL. Stored as SHA-256 hash. Revokable per-session from Admin Panel.
API keys	For scripts and automation. Created in Admin Panel. Shown once. Stored as hash.
Webhook secrets	32-byte random hex, compared using HMAC constant-time comparison (timing-attack safe).

Password Policy

- Minimum 10 characters; at least one uppercase, one lowercase, one digit
- Hashed with bcrypt (12 rounds) — never stored in plaintext
- Admin can force password change on next login per user

Account Lockout

5 consecutive failed login attempts triggers a 15-minute lockout. Configurable via **Admin Panel** → **Security**. Admin can manually unlock immediately from **Admin Panel** → **Users**.

CORS Restriction NEW v14.0

The Flask API enforces Cross-Origin Resource Sharing (CORS) restrictions. Only origins listed in the `ALLOWED_ORIGINS` environment variable are permitted to make cross-origin requests. Default: `http://localhost:8501`.

```
# .env ALLOWED_ORIGINS=http://localhost:8501,https://your-dashboard.company.com
```

Requests from unlisted origins receive a `403 Forbidden` response at the CORS preflight stage, before any authentication is checked.

Rate Limiting NEW v14.0

Endpoint	Limit	Response on breach
Agent <code>/run</code>	30 / minute per IP	429 Too Many Requests
Agent <code>/stream</code>	20 / minute per IP	429 Too Many Requests
Workflow <code>/trigger</code>	10 / minute per IP	429 Too Many Requests

Security Response Headers NEW v14.0

All API responses include the following security headers:

Header	Value
X-Content-Type-Options	<code>nosniff</code>
X-Frame-Options	<code>DENY</code>
X-XSS-Protection	<code>1; mode=block</code>

Namespace Isolation

Client and Viewer role users are hard-scoped to their assigned namespace. Every API endpoint enforces namespace access in the application layer — a client user cannot read, write, or search memory from another namespace regardless of API parameters sent.

Namespace KPI Scoping NEW v14.0

Overview KPIs and Usage dashboard data are now filtered by the logged-in user's namespace for client and viewer roles. Admin and Operator continue to see global counts. This prevents information leakage across namespaces in shared dashboard views.

Audit Log

Every significant action is recorded: login success/failure, logout, token refresh, user creation/deactivation/password reset, API key creation/revocation.

View in **Admin Panel** → **Audit Log**. Each entry records event type, username, IP address, user agent, and timestamp.

Admin Panel Controls

Control	What It Does
Users	Create, deactivate, unlock, and reset passwords.
API Keys	Create and revoke API keys. Raw key shown once.
Sessions	View all active refresh token sessions. Revoke any immediately.
Security Config	Lockout threshold/duration, token TTLs, self-registration toggle.
Audit Log	Paginated full event log with filtering.
Branding	Per-namespace company name, colours, icon, background (see Sec 14).

What ClaudeOS Does NOT Store

- Plaintext passwords — only bcrypt hashes
- Raw refresh tokens — only SHA-256 hashes
- Raw API keys beyond the initial response — only hashes

- Anthropic API key in the database — only in the server's `.env`

19 — Cloud Sync (Supabase)

Overview

ClaudeOS can sync outputs and memory entries to a Supabase PostgreSQL database for cloud backup and off-site storage. Sync is push-only — SQLite on your server is always the source of truth. Supabase is a mirror.

Setup

1. Create a free account at supabase.com and create a new project
2. In the Supabase SQL Editor, run `sync/supabase_schema.sql`
3. Copy your Supabase project URL and service role key from project settings (API section)

Add to `.env`:

```
SUPABASE_URL=https://your-project.supabase.co SUPABASE_SERVICE_KEY=your_service_role_key
```

5. Restart the server — auto-sync starts, firing every 15 minutes

Manual Sync

Go to **Settings** → click **Sync Now** to push immediately.

What Is Synced

- Memory entries
- Agent runs and outputs
- Namespaces and projects
- System events

Note: Auth data (users, sessions, passwords) is *not* synced to Supabase. It stays on your server only.

20 — Troubleshooting & Support

Common Problems and Fixes

Problem	Solution
Cannot reach the dashboard	Run <code>.\scripts\start.ps1</code> . Check ports 5000 and 8501. Verify no firewall blocks them.
Login fails	Login is case-insensitive for username. After 5 failures, wait 15 minutes or ask admin to unlock.
“Must change password” shown	Enter current password then your new password in the prompt.
API returns 401 Unauthorized	Access token expired. Log out and back in. If using API key, verify it is not revoked.
API returns 403 Forbidden	Check <code>ALLOWED_ORIGINS</code> in <code>.env</code> includes your dashboard URL (CORS).
API returns 429 Too Many Requests	You have exceeded the rate limit. Wait 60 seconds before retrying.
Agent runs are slow	Expected for long responses (~40 tokens/second). Check latency in Observability.
Eval scores not appearing	Wait 10–15 seconds after run completes. Scores are async. Check logs for Haiku eval errors.
Streaming returns an error	Ensure only one Flask process is running on port 5000. Restart with <code>.\scripts\start.ps1</code> .
Voice input not working	Run <code>pip install openai-whisper</code> . First use downloads ~140 MB — wait for completion.
MCP server not found	Run <code>pip install mcp uvicorn</code> then <code>.\scripts\start_mcp.ps1</code> .
Runs stuck as “running”	Restart the server. Stuck runs are auto-cleaned on the next health check.
Agents not responding	Check <code>ANTHROPIC_API_KEY</code> in <code>.env</code> . Verify not expired in Anthropic console.
ChromaDB slow on first start	Normal — sentence-transformers model loads on first request (30–60 second cold start).
Ticket email notifications not sending	Verify SMTP settings in Settings → Email Notifications. Use Test Send button to debug. Gmail requires an App Password, not your account password.
Ticket comments not loading	Click the Comments toggle — comments load on demand.

Supabase sync fails	Verify <code>SUPABASE_URL</code> and <code>SUPABASE_SERVICE_KEY</code> . Restart after changes.
Output PDF export fails	Try Markdown export instead. PDF export requires wkhtmltopdf installed on the server.
Memory entries not in agent context	Entries may be archived or expired. Check Memory page with all filters cleared. Verify namespace matches between memory write and agent run.
Namespace branding not appearing	Confirm branding was saved in Admin → Branding. Log out and back in to force a brand refresh.
Background colour not applying	Ensure “Use custom background” checkbox is checked in sidebar expander. Note: session-only — resets on logout.
Onboarding modal not showing	Modal only shows once per account (persisted via <code>onboarding_done</code> DB column, migration 018). Admin accounts skip the onboarding tour. If the tour should re-appear, reset with: <code>UPDATE users SET onboarding_done=0 WHERE username='...'</code>
Onboarding shows every login (not persisting)	Migration 018 may not be applied. Run <code>python scripts/migrate.py</code> . Also verify <code>POST /auth/onboarding-done</code> is called on skip/complete in <code>dashboard/components/onboarding.py</code> .
Voice input widget not clearing after conversation reset	The <code>st.audio_input</code> session state key must be reset alongside conversation history. Verify <code>dashboard/_pages/_agents.py</code> clears the audio key on Clear Conversation click.
Images not received by agent in chat	The SSE stream endpoint must forward the <code>images=</code> parameter to <code>execute_stream()</code> . Verify <code>core/api/routes/agents.py</code> <code>stream_agent()</code> passes images through.
Admin Unlock button always visible (not context-aware)	The Unlock button should only appear when the user is actually locked. Verify <code>dashboard/_pages/_admin.py</code> checks <code>locked_until</code> before rendering the button.
Namespace Usage tab not showing data	Requires at least one agent run to exist. Zero values are normal for new deployments. Tab is only visible to Admin and Operator roles.
Client Onboarding tab missing in Admin Panel	Run <code>python scripts/seed_client_schema.py --namespace <slug></code> to seed blank fields. The tab is in Admin Panel (6th tab) — visible to Admin role only.
Page not preserved after browser refresh	Ensure you are on the latest v14.0. The URL should show <code>?page=PageName</code> . Clear browser cache if the behaviour persists.
Pulse Score shows “Getting Started”	Normal for new namespaces. The score appears once there are agent runs, tickets, and memory entries.

Contact Your Administrator

Detail	Value
--------	-------

Name	[ADMIN NAME]
Email	[ADMIN EMAIL]
Phone	[ADMIN PHONE]
Dashboard URL	[http://YOUR-SERVER-IP:8501]

Before Contacting Support

1. Check this section for your exact error
2. Note the error message shown in the dashboard or browser
3. Note what you were doing (which page, which agent, which prompt)
4. Note whether the error is reproducible
5. Check Observability for any system-wide alerts

Useful Information to Provide

- Your username and role
- Your namespace
- The run ID (visible in Run History) if the issue is with a specific agent run
- Approximate time the issue occurred